

EXPERIMENT 10: Function Template & Class Template

1. Aim

To write C++ programs to understand the implementation of Function Templates and Class Templates.

2. Objectives

To study the concept of generic programming.

To learn how templates reduce code redundancy.

To demonstrate the ability to write and use function templates and class templates.

3. Introduction

Templates are one of the most powerful features of C++. They support **generic programming**, which enables writing a single piece of code that works with different data types. Instead of defining multiple versions of the same function or class, templates allow creating one generic definition. During compilation, the compiler automatically generates type-specific code based on the data type used in the program.

4. Theory

4.1 Function Templates

Function templates provide a way to parameterize data types within a function. They help in creating a function skeleton that can work with any datatype.

Example Use Cases:

Comparison functions

Sorting algorithms

Swapping values

General Syntax:

```
template <typename T>T functionName(T a, T b) {  
    // function body  
}
```

4.2 Class Templates

Class templates allow writing generic classes that work with any data type. They are widely used to implement data structures like Stack, Queue, LinkedList, etc.

General Syntax:

```
template <class T>class ClassName {  
    T var;public:  
    void function(T x);  
};
```

4.3 Advantages of Templates

- Eliminates code repetition
- Cost-efficient and time-saving
- Type-safe generic programming
- Allows creation of reusable components

5. Algorithm

Function Template Algorithm

- Start the program.
- Define a function template with type variable T.
- Write logic inside the function using generic type.
- Call the function with various data types.
- Display output.
- End.

Class Template Algorithm

- Start the program.
- Define a class template with type T.
- Create constructors and member functions.
- Create class objects using different data types.
- Call functions and display results.

End

6. Program

A. Function Template Program

```
#include <iostream>using namespace std;
template <class T>T findMax(T a, T b) {
    return (a > b) ? a : b;
}
int main() {
    cout << "Max of 10 and 20 = " << findMax(10, 20) << endl;
    cout << "Max of 3.4 and 6.8 = " << findMax(3.4, 6.8) << endl;
    cout << "Max of 'A' and 'Z' = " << findMax('A', 'Z') << endl;
    return 0;
}
```

Expected Output

Max of 10 and 20 = 20
Max of 3.4 and 6.8 = 6.8
Max of A and Z = Z

B. Class Template Program

```
#include <iostream>using namespace std;
template <class T>class Calculator {
    T num1, num2;public:
    Calculator(T a, T b) { num1 = a; num2 = b; }

    T add() { return num1 + num2; }
    T subtract() { return num1 - num2; }
    T multiply() { return num1 * num2; }
};
int main() {
    Calculator<int> C1(8, 4);
    Calculator<float> C2(5.5, 1.5);

    cout << "Integer Addition: " << C1.add() << endl;
    cout << "Integer Subtraction: " << C1.subtract() << endl;
    cout << "Float Multiplication: " << C2.multiply() << endl;

    return 0;
}
```

Expected Output

Integer Addition: 12Integer Subtraction: 4Float Multiplication: 8.25

7. Applications

Generic data structures

Mathematical and scientific computations

Libraries like STL use extensive templates

8. Viva Questions (Extended)

What is generic programming?

→ A style of programming where algorithms are written in terms of types to-be-specified later.

Difference between class and function templates?

→ Function templates work with functions; class templates work with classes and objects.

What is template instantiation?

→ Process of generating type-specific code from templates.

Can templates be overloaded?

→ Yes.

Which library uses templates heavily?

→ Standard Template Library (STL).

EXPERIMENT 11: Exception Handling Techniques

1. Aim

To write a C++ program to understand various exception handling techniques using try, throw, and catch.

2. Objectives

To handle runtime errors gracefully.

To prevent program termination due to errors.

To understand multiple catch blocks and catching different data types.

3. Introduction

Exception handling in C++ is used to detect and respond to runtime errors. These errors may occur due to invalid input, memory access violations, division by zero, or file handling errors. Exception handling prevents program crashes and improves user experience.

4. Theory

4.1 Keywords

try – Code that may produce an exception

throw – Signals an exception

catch – Handles the thrown exception

4.2 Types of Exceptions

Built-in exceptions

User-defined exceptions

4.3 Multiple Catch Blocks

Used to handle different types of errors.

5. Algorithm

Start the program.

Input two numbers.

Place division operation inside a try block.

If divisor is zero, throw an exception.

Catch the exception using catch block.

Display error message.

End.

6. Program

```
#include <iostream>using namespace std;
int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;

    try {
        if (b == 0)
            throw runtime_error("Error: Division by zero is not allowed!");

        cout << "Result = " << a / b << endl;
    }
    catch (runtime_error &e) {
        cout << "Exception Caught: " << e.what() << endl;
    }

    return 0;
}
```

Expected Output

Enter two numbers: 12 0

Exception Caught: Error: Division by zero is not allowed!

7. Advantages

Prevents abnormal termination

Provides meaningful error messages

Enhances reliability and security

8. Applications

Banking and transactional applications

File and memory intensive programs

Database systems

9. Viva Questions (Extended)

What is an exception?

→ A runtime error/abnormal condition.

What happens if an exception is not caught?

→ The program terminates immediately.

Can we have a try without catch?

→ Yes, only if it is followed by a `throw` and handled elsewhere.

What is `e.what()`?

→ A function that returns exception description.

Difference between `throw` and `catch`?

→ `throw` raises an error; `catch` handles it.

EXTENDED EXPERIMENT 12: File Handling Techniques

1. Aim

To write a C++ program to understand file reading and writing operations using `ifstream`, `ofstream`, and `fstream`.

2. Objectives

To study file streams in C++.

To read and write data using files.

To understand how data is stored persistently.

3. Introduction

File handling is an essential part of programming that allows storing data permanently on disk. C++ provides classes in `<fstream>` for performing operations like creating files, writing data, reading data, and closing files.

4. Theory

4.1 File Stream Classes

Class	Purpose
ifstream	Read from files
ofstream	Write to files
fstream	Read & Write

4.2 Common Functions

`open()` – Opens file

`close()` – Closes file

`getline()` – Reads a full line

`<<` – Write operator

`>>` – Read operator

5. Algorithm

Start program.

Create a file using ofstream.

Write text into file.

Close the file.

Open file using ifstream.

Read file line-by-line using getline().

Display the output.

Close the file.

End.

6. Program

```
#include <iostream>#include <fstream>using namespace std;
int main() {

    // Writing to file
    ofstream fout("student.txt");
    fout << "Name: Kanchan Prajapat\n";
    fout << "Course: B.Tech Computer Science\n";
    fout << "Subject: File Handling Experiment\n";
    fout.close();

    // Reading from file
    ifstream fin("student.txt");
    string data;

    cout << "----- Reading File Content ----- \n";
    while (getline(fin, data)) {
        cout << data << endl;
    }
    fin.close();

    return 0;
}
```

Expected Output

```
----- Reading File Content -----
Name: Kanchan Prajapat
```

7. Advantages

- Permanent storage
- Can store large data
- Easy data retrieval

8. Applications

- Student databases
- Billing systems
- Log file management
- Document storage systems

9. Viva Questions (Extended)

Which header file is used for file handling?

→ <fstream>

What is the difference between ifstream and ofstream?

→ ifstream is used for reading; ofstream for writing.

What is the use of getline()?

→ Reads an entire line from file.

What happens if a file is not found?

→ The open() function fails.

Why file handling is needed?

→ To store data permanently.